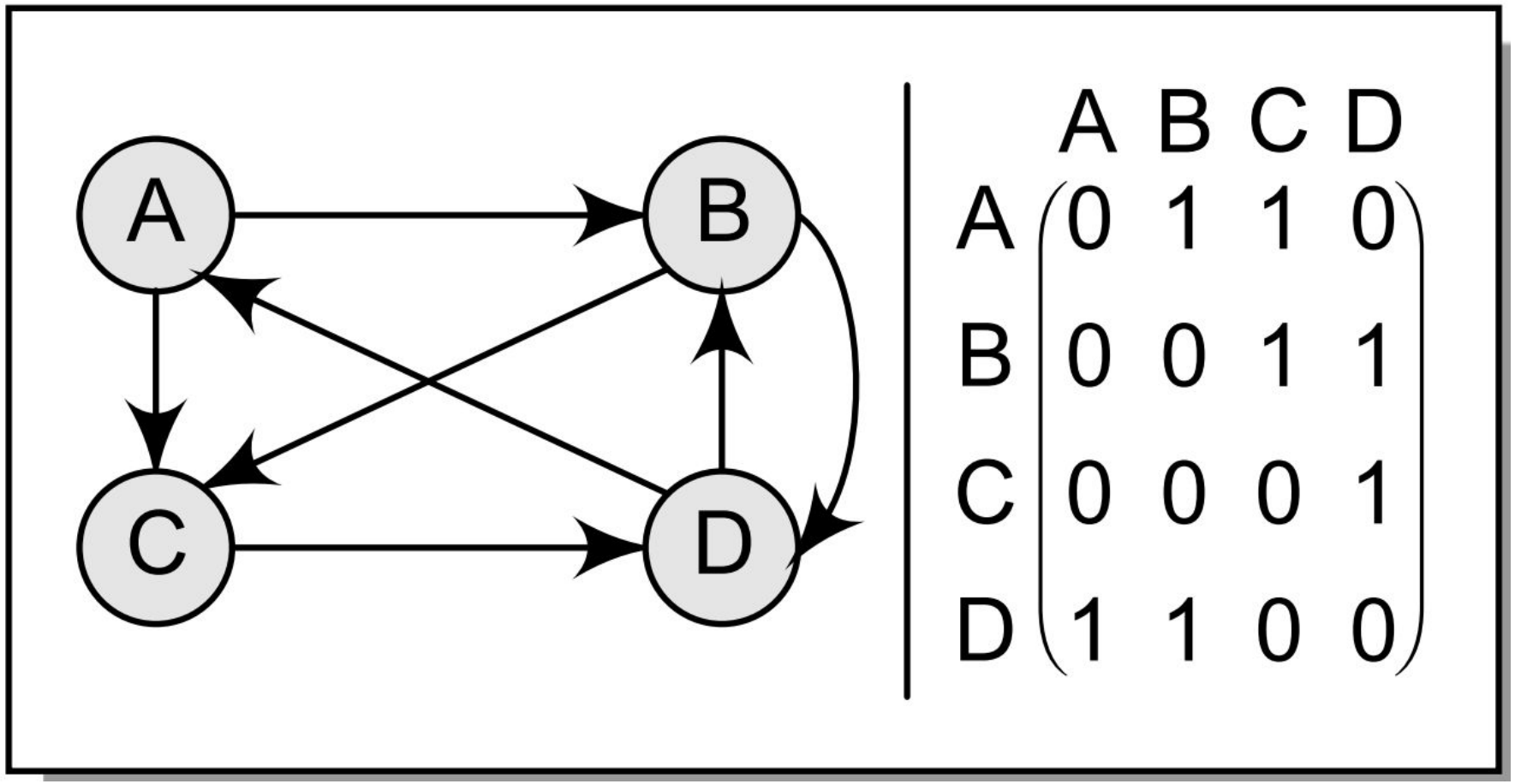
Floyd-Warshall Algorithm

Adjacency Matrix



	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	1
D	1	1	0	0

Path Matrix

$$A^2 = A^1 \times A^1$$

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 4 & 2 \\ 1 & 2 & 3 & 4 \\ 1 & 1 & 1 & 3 \\ 3 & 4 & 3 & 4 \end{bmatrix}$$

Page: 389 (Data structure using C)

Path Matrix

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 2 & 1 \end{bmatrix} + \begin{bmatrix} 2 & 2 & 0 & 1 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 2 & 1 \\ 1 & 2 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 1 & 3 & 4 & 2 \\ 1 & 2 & 3 & 4 \\ 1 & 1 & 1 & 3 \\ 3 & 4 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 & 5 \\ 3 & 5 & 6 & 7 \\ 2 & 3 & 3 & 5 \\ 6 & 8 & 7 & 8 \end{bmatrix}$$

Now the path matrix P can be given as:

$$P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Warshall's Algorithm

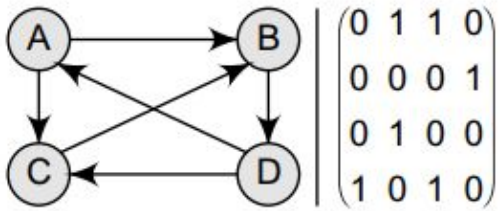
$$P_k[i][j] = P_{k-1}[i][j] \vee (P_{k-1}[i][k] \wedge P_{k-1}[k][j])$$

Floyd-Warshall Algorithm

Algorithm 1: Pseudocode of Floyd-Warshall Algorithm

```
Data: A directed weighted graph G(V, E)
Result: Shortest path between each pair of vertices in G
for each d ∈ V do
    distance[d][d] ← 0;
end
for each edge (s, p) ∈ E do
    distance[s][p] ← weight(s, p);
end
n = cardinality(V);
for k = 1 to n do
    for i = 1 to n do
        for j = 1 to n do
            if distance[i][j] > distance[i][k] + distance[k][j] then
                distance[i][j] ← distance[i][k] + distance[k][j];
            end
        end
    end
end
```

Floyd-Warshall Algorithm



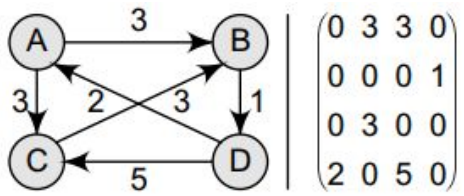
$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Figure 13.41 Graph G

$$\mathbf{Q0} = \begin{bmatrix} 9999 & 1 & 1 & 9999 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 1 & 9999 & 9999 \\ 1 & 9999 & 1 & 9999 \end{bmatrix} \quad \mathbf{Q1} = \begin{bmatrix} 9999 & 1 & 1 & 9999 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 1 & 9999 & 9999 \\ 1 & 2 & 1 & 9999 \end{bmatrix}$$

$$\mathbf{Q2} = \begin{bmatrix} 9999 & 1 & 1 & 2 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 1 & 9999 & 2 \\ 1 & 2 & 9999 & 3 \end{bmatrix} \quad \mathbf{Q3} = \begin{bmatrix} 9999 & 1 & 1 & 2 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 1 & 9999 & 2 \\ 1 & 2 & 9999 & 3 \end{bmatrix} \quad \mathbf{Q4} = \mathbf{Q} \begin{bmatrix} 3 & 1 & 1 & 2 \\ 2 & 3 & 2 & 1 \\ 3 & 1 & 3 & 2 \\ 1 & 2 & 1 & 3 \end{bmatrix}$$

Floyd-Warshall Algorithm



$$\begin{pmatrix} 0 & 3 & 3 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 3 & 0 & 0 \\ 2 & 0 & 5 & 0 \end{pmatrix}$$

Figure 13.42 Graph G

$$\mathbf{Q0} = \begin{bmatrix} 9999 & 3 & 3 & 9999 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 3 & 9999 & 9999 \\ 2 & 9999 & 5 & 9999 \end{bmatrix}$$

$$\mathbf{Q1} = \begin{bmatrix} 9999 & 3 & 3 & 9999 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 3 & 9999 & 9999 \\ 2 & 5 & 5 & 9999 \end{bmatrix}$$

$$\mathbf{Q2} = \begin{bmatrix} 9999 & 3 & 3 & 4 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 3 & 9999 & 4 \\ 2 & 5 & 5 & 6 \end{bmatrix}$$

$$\mathbf{Q3} = \begin{bmatrix} 9999 & 3 & 3 & 4 \\ 9999 & 9999 & 9999 & 1 \\ 9999 & 3 & 9999 & 4 \\ 2 & 5 & 5 & 6 \end{bmatrix}$$

$$\mathbf{Q4} = \mathbf{Q}\begin{bmatrix} 6 & 3 & 3 & 4 \\ 3 & 6 & 6 & 1 \\ 6 & 3 & 9 & 4 \\ 2 & 5 & 5 & 6 \end{bmatrix}$$

Proof of correctness

- Let G = (V, E), with |V| = n
- w(i, j): weight of edge (i, j), or ∞ if no edge
- $d^{(k)}[i][j]$: shortest path from i to j using intermediate vertices in {1,...,k}

Base Case (k = 0)

$$d^{(0)}[i][j] = \begin{cases} 0 \text{ if } i = j \\ w(i, j) \text{ if } (i, j) \in E \\ \infty \text{ otherwise} \end{cases}$$

This matches the shortest path from i to j with **no intermediate vertices** (i.e., either direct edge, or nothing).

Inductive Step

- Assume invariant holds for k – 1 iteration:

$$\text{for all } i, j,\ d[i][j] = d^{(k-1)}[i][j]$$

- At k iteration:

$$d^{(k)}[i][j] = \min\left(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\right)$$

Inductive Step

Two different cases:

1. Path doesn't use k → use existing

$$d^{(k-1)}[i][j].$$

2. Path uses k → combine $d^{(k-1)}[i][k]$ and $d^{(k-1)}[k][j]$.

$$d^{(k-1)}[i][k] + d^{(k-1)}[k][j]$$

After n iterations, all vertices allowed as intermediates

Practice Problems

- https://www.spoj.com/problems/ROHAAN/
- https://www.spoj.com/problems/SOCIALNE/